BACKEND SCHEMA REFERENCE

ConnectLocal

Complete Mongoose data-model source — Users, Connections, Messages, Reports, controlled vocabulary, and DB setup, matching the TRD v1.0 specification.

Version 1.0 · MVP Scope

Companion to: ConnectLocal PRD v1.0 & TRD v1.0

Contents

1	**User Schema**	`models/User.js`
2	**Connection Schema**	`models/Connection.js`
3	**Message Schema**	`models/Message.js`
4	**Report Schema**	`models/Report.js`
5	**Models Barrel File**	`models/index.js`
6	**Skills & Interests Vocabulary**	`config/vocabulary.js`
7	**Database Connection**	`db/connect.js`

Stack: Node.js + Express + MongoDB (Mongoose ODM). All schemas validated to load and enforce their constraints (age-gate, controlled skills/interests vocabulary, unique connection pairs) before inclusion in this document.

01 User Schema

models/User.js

Profile fields, age-gate validator, skills/interests, indexes.

```
const mongoose = require("mongoose");
const { Schema } = mongoose;
const { ALL_SKILLS, ALL_INTERESTS, INTENTS } = require("../config/vocabulary");

const MIN_AGE = 18;

const UserSchema = new Schema(
  {
    // External auth provider UID (Firebase Auth / Clerk)
    authId: {
      type: String,
      required: true,
      unique: true,
      index: true,
    },

    name: {
      type: String,
      required: true,
      trim: true,
      maxlength: 60,
    },

    // Required for age-gate enforcement (TRD 7.1). Validated at signup:
    // age = now - dob must be >= MIN_AGE, enforced in the profile module,
    // not just here — this validator is a defense-in-depth backstop.
    dob: {
      type: Date,
      required: true,
      validate: {
        validator: function (value) {
          const ageInMs = Date.now() - value.getTime();
          const ageInYears = ageInMs / (1000 * 60 * 60 * 24 * 365.25);
          return ageInYears >= MIN_AGE;
        },
        message: `User must be at least ${MIN_AGE} years old.`,
      },
    },

    gender: {
      type: String,
      required: true,
      enum: ["male", "female", "non_binary", "prefer_not_to_say"],
    },

    // Hard filter for Discover (TRD Section 5). Always normalize casing on write
    // (e.g. "delhi" -> "Delhi") in the profile service before save.
    city: {
      type: String,
      required: true,
      trim: true,
      index: true,
    },

    // Reserved for future neighborhood-level granularity (e.g. "Connaught Place").
    // Not surfaced in MVP UI or used in any query yet — kept so no migration
    // is needed later.
    area: {
      type: String,
      trim: true,
      default: null,
    },

    // Cloudinary secure URL. Only ever set AFTER moderation passes
    // (TRD Section 7.3) — never write an unmoderated URL here.
    photoUrl: {
      type: String,
      default: null,
    },

    about: {
      type: String,
      trim: true,
      maxlength: 300,
      default: "",
    },

    // Professional tags — controlled vocabulary only, no free text.
    skills: {
      type: [String],
      enum: ALL_SKILLS,
      default: [],
      validate: {
```

```
        validator: (arr) => arr.length <= 10,
        message: "A maximum of 10 skills is allowed.",
      },
    },

    // Hobby tags — controlled vocabulary only, no free text.
    interests: {
      type: [String],
      enum: ALL_INTERESTS,
      default: [],
      validate: {
        validator: (arr) => arr.length <= 10,
        message: "A maximum of 10 interests is allowed.",
      },
    },

    intent: {
      type: [String],
      enum: INTENTS,
      required: true,
      validate: {
        validator: (arr) => arr.length >= 1,
        message: "At least one intent must be selected.",
      },
    },

    status: {
      type: String,
      enum: ["active", "soft_deleted", "hidden_pending_review", "banned"],
      default: "active",
      index: true,
    },

    // Resets daily via scheduled job or lazy check in the rate-limit
    // middleware (TRD Section 6.2). Not the source of truth for enforcement
    // by itself — paired with a rolling-window check in middleware.
    dailyConnectCount: {
      type: Number,
      default: 0,
    },
    dailyConnectCountResetAt: {
      type: Date,
      default: Date.now,
    },

    lastActiveAt: {
      type: Date,
      default: Date.now,
      index: true,
    },

    // Soft-delete bookkeeping (TRD Section 7.5)
    deletedAt: {
      type: Date,
      default: null,
    },

    // Consent timestamp for Terms/Privacy Policy (TRD Section 7.6)
    consentAcceptedAt: {
      type: Date,
      default: null,
    },
  },
  {
    timestamps: true, // adds createdAt / updatedAt
  }
);

// Compound indexes powering Discover Tier 1 / Tier 2 queries (TRD 4.6)
UserSchema.index({ city: 1, skills: 1 });
UserSchema.index({ city: 1, interests: 1 });

// Never expose these fields in API responses to OTHER users (only to self/admin).
UserSchema.methods.toPublicProfile = function () {
  return {
    id: this._id,
    name: this.name,
    city: this.city, // city-level only — never expose precise location (TRD 7.4)
    photoUrl: this.photoUrl,
    about: this.about,
    skills: this.skills,
    interests: this.interests,
    intent: this.intent,
  };
};

module.exports = mongoose.models.User || mongoose.model("User", UserSchema);
```

170 lines

02 Connection Schema

models/Connection.js

Mutual-connect state machine + sorted-pair uniqueness.

```
const mongoose = require("mongoose");
const { Schema } = mongoose;

const ConnectionSchema = new Schema(
  {
    // Always stored sorted so userA < userB (compare _id as strings) at the
    // application layer before save. This guarantees exactly one document
    // per pair regardless of who sends the Connect request.
    userA: {
      type: Schema.Types.ObjectId,
      ref: "User",
      required: true,
      index: true,
    },
    userB: {
      type: Schema.Types.ObjectId,
      ref: "User",
      required: true,
      index: true,
    },

    // Which of userA/userB actually sent the original Connect request.
    initiatedBy: {
      type: Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },

    status: {
      type: String,
      enum: ["pending", "accepted", "ignored", "unmatched", "blocked"],
      default: "pending",
      index: true,
    },

    respondedAt: {
      type: Date,
      default: null,
    },
  },
  {
    timestamps: true, // createdAt = when Connect was sent
  }
);

// Prevents duplicate connection documents for the same pair.
// IMPORTANT: application code must always write userA/userB in sorted
// order (e.g. userA = smaller ObjectId) for this to correctly dedupe
// regardless of who initiates.
ConnectionSchema.index({ userA: 1, userB: 1 }, { unique: true });

// "My connections by status" lookups (TRD 4.6)
ConnectionSchema.index({ userA: 1, status: 1 });
ConnectionSchema.index({ userB: 1, status: 1 });

/**
 * Helper: given a connection doc and one of the two user ids in it,
 * returns the OTHER user's id. Used heavily by the Tier 3 (second-degree
 * network suggestion) matching logic in TRD Section 5.3.
 */
ConnectionSchema.statics.otherUserId = function (connectionDoc, knownUserId) {
  const known = knownUserId.toString();
  return connectionDoc.userA.toString() === known
    ? connectionDoc.userB
    : connectionDoc.userA;
};

/**
 * Helper: build a sorted [userA, userB] pair so callers never have to
 * remember the sort-order invariant themselves.
 */
ConnectionSchema.statics.sortedPair = function (idOne, idTwo) {
  const [a, b] = [idOne.toString(), idTwo.toString()].sort();
  return { userA: a, userB: b };
};

module.exports =
  mongoose.models.Connection || mongoose.model("Connection", ConnectionSchema);
```

78 lines

03 Message Schema

models/Message.js

Chat messages, keyed by connectionId.

```
const mongoose = require("mongoose");
const { Schema } = mongoose;

const MessageSchema = new Schema(
  {
    connectionId: {
      type: Schema.Types.ObjectId,
      ref: "Connection",
      required: true,
      index: true,
    },

    senderId: {
      type: Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },

    // MVP is text-only — no voice/video (per PRD Section 8 / TRD scope).
    text: {
      type: String,
      required: true,
      trim: true,
      maxlength: 2000,
    },

    readAt: {
      type: Date,
      default: null,
    },
  },
  {
    timestamps: { createdAt: true, updatedAt: false },
  }
);

// Fetching a thread's history in order (TRD 4.6)
MessageSchema.index({ connectionId: 1, createdAt: 1 });

/**
 * Application-layer rule (enforce in the chat service, not just here):
 * a message can only be created if the parent Connection.status === "accepted".
 * Mongoose alone cannot cross-validate another collection at the schema
 * level, so this must be checked in the POST /connections/:id/messages
 * handler before calling Message.create(...).
 */

module.exports = mongoose.models.Message || mongoose.model("Message", MessageSchema);
```

48 lines

04 Report Schema

models/Report.js

Abuse reports + auto-hide threshold.

```
const mongoose = require("mongoose");
const { Schema } = mongoose;

const ReportSchema = new Schema(
  {
    reporterId: {
      type: Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },

    reportedUserId: {
      type: Schema.Types.ObjectId,
      ref: "User",
      required: true,
      index: true,
    },

    reason: {
      type: String,
      enum: ["spam", "fake_profile", "harassment", "inappropriate_photo", "other"],
      required: true,
    },

    details: {
      type: String,
      trim: true,
      maxlength: 500,
      default: "",
    },

    status: {
      type: String,
      enum: ["open", "reviewed_no_action", "actioned"],
      default: "open",
      index: true,
    },
  },
  {
    timestamps: { createdAt: true, updatedAt: false },
  }
);

// Powers the auto-hide rule (TRD 7.2): count open reports for a user.
ReportSchema.index({ reportedUserId: 1, status: 1 });

/**
 * Auto-hide threshold, referenced by the trust-safety service after
 * a new report is created (TRD Section 7.2):
 *
 *   const openCount = await Report.countDocuments({
 *     reportedUserId: X, status: "open"
 *   });
 *   if (openCount >= Report.AUTO_HIDE_THRESHOLD) {
 *     await User.updateOne({ _id: X }, { status: "hidden_pending_review" });
 *   }
 */
ReportSchema.statics.AUTO_HIDE_THRESHOLD = 3;

module.exports = mongoose.models.Report || mongoose.model("Report", ReportSchema);
```

60 lines

05 Models Barrel File

models/index.js

Single import point for all models.

```
const User = require("./User");
const Connection = require("./Connection");
const Message = require("./Message");
const Report = require("./Report");

module.exports = { User, Connection, Message, Report };
```

6 lines

06 Skills & Interests Vocabulary

config/vocabulary.js

Controlled tag list used by the User schema.

```
/**
 * Controlled vocabulary for skills & interests.
 *
 * Per TRD Section 4.5: skills/interests are NOT free text. They are selected
 * from this fixed list at signup so that Discover matching (city + skills/interests)
 * can use exact-match queries instead of fuzzy text matching.
 *
 * To add a new tag: add it to the relevant category array below.
 * Both SKILLS and INTERESTS are flattened into single arrays for schema
 * enum validation, but kept grouped here for the UI to render by category.
 */

const SKILLS_BY_CATEGORY = {
  Tech: ["AI", "Backend Dev", "Frontend Dev", "Design", "Data", "Product"],
  Business: ["Marketing", "Sales", "Finance", "Operations"],
  Creative: ["Writing", "Video", "Photography"],
};

const INTERESTS_BY_CATEGORY = {
  Fitness: ["Gym", "Running", "Yoga"],
  Sports: ["Football", "Cricket", "Badminton"],
  Lifestyle: ["Travel", "Food", "Music", "Reading"],
  Gaming: ["PC Gaming", "Console Gaming", "Mobile Gaming"],
};

function flatten(byCategory) {
  return Object.values(byCategory).reduce((acc, list) => acc.concat(list), []);
}

const ALL_SKILLS = flatten(SKILLS_BY_CATEGORY);
const ALL_INTERESTS = flatten(INTERESTS_BY_CATEGORY);

const INTENTS = [
  "friendship",
  "networking",
  "startup",
  "sports",
  "gaming",
  "hobbies",
  "dating",
  "study",
];

module.exports = {
  SKILLS_BY_CATEGORY,
  INTERESTS_BY_CATEGORY,
  ALL_SKILLS,
  ALL_INTERESTS,
  INTENTS,
};
```

50 lines

07 Database Connection

db/connect.js

MongoDB (Atlas) connection setup.

```
const mongoose = require("mongoose");

/**
 * Connects to MongoDB (Atlas) using MONGODB_URI from environment.
 * Call this once at server startup before mounting routes.
 */
async function connectDB() {
  const uri = process.env.MONGODB_URI;
  if (!uri) {
    throw new Error("MONGODB_URI is not set in environment variables.");
  }

  mongoose.set("strictQuery", true);

  await mongoose.connect(uri, {
    // Mongoose 7+/8+ no longer needs useNewUrlParser/useUnifiedTopology,
    // they're defaults. Left here as a reminder if using an older driver.
  });

  console.log("[db] MongoDB connected");

  mongoose.connection.on("error", (err) => {
    console.error("[db] MongoDB connection error:", err);
  });

  return mongoose.connection;
}

module.exports = { connectDB };
```

29 lines